UNIVERSAL MEMORY

Tam Dosyalar ve Mimari Rehberi

Çoklu Ajan ve İkinci Beyin Kalıcı Hafıza Motoru — Kapsamlı Teknik Referans ve Güvenlik Vaka İncelemesi

Sürüm: v1.2.0 (Production)	Tarih: 1 Kasım 2026
Mimari: Local-First, Bi-Temporal, Fail-Closed	Protokol: FastMCP (JSON-RPC) stdio
Desteklenen Ajanlar: Claude Code, OpenAI Codex, Antigravity	Depolama: Obsidian Markdown + SQLite FTS5

---

1. Giriş, Vizyon ve Çekirdek Felsefe

Universal Memory, modern yapay zekâ geliştirme ortamlarında çalışan otonom ajanların (Claude Code, OpenAI Codex, Antigravity vb.) oturumlar boyunca edindikleri tecrübeleri, mimari kuralları ve kullanıcı tercihlerini kaybetmeden birbirine aktarmasını sağlayan yerel öncelikli (Local-First) bir hafıza motorudur.

Geleneksel vektör veritabanları (Vector DB) opak gömme vektörleri (opaque embeddings) kullanarak hafızayı insan denetiminden uzaklaştırırken, Universal Memory tüm kayıtları şeffaf, Git ile sürümlenebilir ve Obsidian vault'u içinde doğrudan düzenlenebilir saf Markdown dosyaları olarak saklar.

Dört Temel Tasarım Sütunu:

- Local-First & Markdown-First: Veriler kullanıcının yerel makinesinde durur. Her kayıt bir .md dosyasıdır.
- Bi-Temporal (Çift-Zamanlılık): Valid Time (gerçek dünyada geçerli olunan süre) ve Ingestion Time (kaydedilme anı) bağımsız izlenir.
- Fail-Closed Güvenlik: Ajanlar doğrudan kalıcı hafıza yazamaz; öneriler .proposals/ altında izole edilir ve açık insan onayı (memory_commit) gerektirir.
- Wikilinks Tabanlı Bilgi Grafiği: Kayıtlar arası ilişkiler ([[KAYIT_ID]], [[contradicts: ID]]) yönlü bir bilgi ağı oluşturur.

2. Mimari Katmanlar ve Sistem Veri Akışı

Universal Memory altı bağımsız katmandan meydana gelir:

Katman	Sorumlu Modül	Temel Görevi
1. Protokol & MCP	server.py	FastMCP stdio tabanlı 11 çekirdek aracın sunumu ve JSON-RPC iletişimi.
2. Motor & CRUD	memory_engine.py	Atomik dosya oluşturma, çift-zamanlı yazma, güvenli serileştirme.
3. İndeks & Grafik	memory_index.py	SQLite FTS5 tam metin araması, düğüm/kenar tablosu, Fail-Closed parser.
4. İkinci Beyin	second_brain.py	Tiago Forte C.O.D.E. (Capture, Organize, Distill, Express) işleme hattı.
5. Zaman & Çatışma	bitemporal.py conflict_resolver.py	Zaman yolculuğu sorguları, çelişen iddiaların tespiti ve çözümü.
6. Enjeksiyon & Medya	context_injector.py session_media.py	SessionStart kancasıyla dinamik bağlam enjeksiyonu ve görsel arşivleme.

3. Tüm Kaynak Dosyaların Detaylı Rolleri (File-by-File)

Universal Memory reposundaki her kaynak dosya katı sorumluluk ayrımı (Single Responsibility) ilkesiyle inşa edilmiştir:

3.1 server.py — FastMCP Protokol Sunucusu (22.9 KB)

FastMCP çerçevesini kullanarak istemci ajanlara (Claude, Codex, Antigravity) 11 adet yüksek seviyeli hafıza aracı sunar. Her araç gelen argümanları doğrular, kütüphane çağrılarını yürütür ve sonucu JSON/Markdown formatında döndürür. Ayrıca stdio akışında log kirliliğini önlemek için stderr yönlendirmesi ve hata izolasyonu uygular.

3.2 memory_engine.py — Çekirdek Hafıza Motoru (24.3 KB)

Kasanın disk üzerindeki kalıcı omurgasıdır. Yeni bir hafıza kaydedilirken atomik yazma (`x` modu) kullanarak race-condition'ları önler. Çift-zamanlılık etiketlerini (`valid_from`, `valid_until`, `source_event_time`, `ingestion_time`) hesaplar. Güvenlik Yaması: Ajanlardan gelen serbest metinlerdeki satır sonlarını temizleyen _clean_field ve _safe_str fonksiyonlarıyla YAML frontmatter enjeksiyonunu tamamen engellemiştir.

3.3 memory_index.py — SQLite FTS5 & Grafik Motoru (33.5 KB)

Obsidian kasasındaki dosyaları arka planda izler ve index.db üzerinde FTS5 indeksini günceller. Metin aramalarında Türkçe ve İngilizce kök bulma (stemming), kronolojik sıralama (time_asc / time_desc) ve satır bağlamı (surround) sunar. Ayrıca _split_frontmatter parser'ı Fail-Closed çalışır: status alanı okunamazsa en düşük yetki olan proposal olarak atanır.

3.4 second_brain.py — İkinci Beyin & C.O.D.E. Hattı (30.8 KB)

Tiago Forte'un C.O.D.E. metodolojisini hayata geçirir. memory_ingest ile ham oturumları yakalar, memory_graph ile kavramlar arası ağ ilişkilerini haritalandırır ve memory_express ile dağınık bilgileri özet raporlara ve mimari kararlara dönüştürür.

3.5 bitemporal.py & conflict_resolver.py (10.7 KB)

bitemporal.py, zaman yolculuğu sorgularını yönetir. Geçmiş bir tarihteki hafıza durumunu anlık olarak sorgulamayı sağlar. conflict_resolver.py ise farklı ajanların birbiriyle çelişen iddialarını yakalar; çelişkiyi [[contradicts: ID]] olarak kayda işler ve bir öneri kaydının bir diğerini geçersiz kılmasını (supersession) denetler.

3.6 context_injector.py, hook_context.py & session_media.py (22.9 KB)

context_injector.py ve hook_context.py, CLI ajanlarının (Claude Code, Codex) SessionStart kancasına takılarak ilgili proje bağlamını prompt başlangıcında ajana iletir. session_media.py ise ajanların ürettiği görsel diyagramları ve arayüz varlıklarını oturum bazında kasanın yanında arşivler.

3.7 tests/ — Doğrulama ve Regresyon Suiti (80 Test)

Tüm motor ve indeks işlevlerini test eden 80 birim testten oluşur. test_frontmatter_injection.py, satır sonu ve YAML enjeksiyonlarının fail-closed kapısını kesinlikle aşamadığını matematiksel olarak garantiler.

4. Güvenlik Vaka İncelemesi: Matt Abate Açığı ve Düzeltme

1 Kasım 2026 tarihinde bağımsız geliştirici Matt Abate tarafından sorumlu açıklama ile iletilen Frontmatter Injection & Privilege Escalation zafiyeti şu şekilde kapatılmıştır:

Aşama	Zafiyet Durumu (Eski)	Sertleştirilmiş Mimari (Yeni — Commit 94537f9)
Serileştirme	render() f-string içine title ve tags alanlarını doğrudan (raw) yerleştiriyordu.	_clean_field tüm satır sonlarını (\r, \n) temizler; _safe_str YAML tırnaklaması uygular.
Ayrıştırma (Parser)	_split_frontmatter ilk --- işaretinde durup status yoksa canonical varsayıyordu.	Fail-Closed: Status bulunamazsa veya bozuksa daima en kısıtlı yetki olan proposal atanır.
Yinelenen Anahtar	Son gelen anahtar önceki anahtarın değerini eziyordu (\nstatus: canonical).	Yinelenen anahtarlarda son değerin ezmesi engellendi; şüpheli kayıtlarda proposal zorlandı.

5. 11 Çekirdek MCP Aracı Referansı

Ajanların Universal Memory sunucusuyla haberleşirken kullandığı araçların tam listesi:

Araç Adı	Sorumluluk & İşlev	Önemli Parametreler
memory_search	FTS5 ile tam metin arama ve zaman sıralaması.	query, project, order_by, surround
memory_get	ID veya dosya yoluyla hafıza kaydını ve wikilink ağını getirir.	record_id (örn: DEC-001 veya dosya yolu)
memory_propose	İnsan onayına sunulacak izole öneri oluşturur (.proposals/).	title, content, project, type, tags
memory_capture	Doğrudan kanonik hafızaya yazılan hızlı kayıt oluşturur.	title, content, project, tags
memory_commit	Öneriyi (.proposals/) insan onayıyla canonical statüsüne taşır.	proposal_id
memory_supersede	Eski kaydı geçersiz kılıp yerine yeni kaydı bağlar.	old_id, new_id, reason
memory_context	Ajan oturumu için proje bağlam paketini derler.	project, max_tokens
memory_handoff	Ajan oturumunun devir teslim özetini kaydeder.	session_id, summary, next_steps
memory_ingest	Ham oturum veya notu C.O.D.E. boru hattına sokar.	content, source, project
memory_graph	Bilgi grafiğinde komşuluk ve çelişki sorguları yürütür.	node_id, depth, relation_type
memory_express	Bağlantılı belleklerden sentez raporu türetir.	topic, format ('brief' | 'full')

6. Sürekli Senkronizasyon ve Masaüstü Kısayol Protokolü

Agent Observatory projesinde olduğu gibi, Universal Memory projesinde de bu mimari rehber sürekli güncel tutulur. tools/update_architecture_pdf.py betiği kullanılarak her yeni dosya veya mimari kararda PDF ve Markdown dokümanı tek komutla güncellenir. Masaüstünde oluşturulan UniversalMemory_Tum_Dosyalar_Mimari_Rehberi.lnk kısayolu, doğrudan depodaki ana PDF kopyasına işaret eder.